哈爾濱工程大學

第 03 讲 / 全 17 讲

C 语言 if-else 语句用法详解

让程序学会做选择：条件成立走这边，不成立走那边

单分支 if

条件成立才执行

例：及格判定

双分支 if-else

二选一必走一支

例：奇偶判断

多路 else if

从上往下第一个成立

例：成绩等级

嵌套与陷阱

花括号决定归属

例：悬空 else

本节目标

哈爾濱工程大學

1. 说出 C 语言里真和假的判定规则：0 为假，非 0 为真
2. 写出 if 单分支程序：条件成立才执行，不成立就跳过
3. 用 if-else 实现二选一，理解必走一支的含义
4. 正确使用 6 个关系运算符，分清比较与赋值
5. 会写嵌套 if 与 else if 阶梯，并知道两者该怎么选
6. 独立完成 5 道判断类编程题：奇偶、最大值、等级、闰年、计费

01

基本语法：从条件说起

本部分讲什么 · if 是什么、真假怎么判、六个关系运算符

哈爾濱工程大学

if 是什么：程序会做选择

前面写的程序都是从上往下逐条执行，一路走到底。 if 的作用是在半路放一个岔路口：条件成立走这边，不成立走那边。

第 1 步

顺序执行

读入数据
一条接一条往下走

第 2 步

条件判断

提出一个是非题
答案只有真或假

第 3 步

选择执行

真：走成立分支
假：走不成立分支

生活里的选择

如果 气温低于 0 度，就 结冰；否则 不结冰。

程序里的选择

```
if (t < 0)
    printf("结冰");        // 低温才打印
```

if 单分支与两条轨迹

哈爾濱工程大學

```
// 单分支：成立就进，不成立就绕
int score;
scanf("%d", &score);
if (score >= 60)
{
    printf(" 及格 ");        // 成立才打印
}
printf(" 程序结束 ");     // 一定会执行
```

条件为真 → 进去

条件成立，执行花括号里的语句，然后顺序往下走。

条件为假 → 跳过

条件不成立，整块花括号被跳过，直接落到后面的语句。

两条轨迹会汇合

不管走哪条路，最后都汇到同一处，后面的语句照常执行。

if-else 双分支：二选一


```
// 双分支：两条路必走一条
int score;
scanf("%d", &score);
if (score >= 60)
{
    printf("及格");      // 60 分及以上
}
else
{
    printf("不及格");    // 60 分以下
}
```

二选一，必走一支

条件成立走 if 块，不成立走 else 块，不会两条都不走，也不会两条都走。

else 不能单独出现

else 必须紧跟在一个 if 的后面，孤零零写一个 else 是语法错误。

else 后面不写条件

剩下的所有情况都归 else，else 后面再加条件就错了。

什么是真：0 假，非 0 真

C 语言没有专门的真和假，它用整数表示：括号里的值算出来是 0 ，条件就不成立；只要不是 0 ，条件就成立。

```
// 括号里不是 0 就算真
int a = 0;                  // a 现在是 0
if (0)      printf("A"); // 不执行
if (1)      printf("B"); // 执行
if (-5)     printf("C"); // 负数也执行
if (100)    printf("D"); // 执行
if (3 > 5)  printf("E"); // 得 0，不执行
if (3 < 5)  printf("F"); // 得 1，执行
if (a)      printf("G"); // a 是 0，不执行
```

0 就是假

只有 0 表示条件不成立，其余一律成立。

非 0 都是真

1 、 -5 、 100 全都算真，负数也不例外。

比较式得 1 或 0

3 大于 5 得 0 ， 3 小于 5 得 1 。

关系运算符六兄弟

条件怎么写？靠关系运算符把两个数比一比，比完的结果是 1（成立）或 0（不成立）。

红线：== 是比较两边是否相等，= 是把右边存进左边。写反了程序不报错，但结果全错。

==
等于
`a == b`

!=
不等于
`a != b`

>
大于
`a > b`

<
小于
`a < b`

>=
大于等于
`a >= b`

<=
小于等于
`a <= b`

最简单的判断程序

哈爾濱工程大學

任务 读入一个整数，如果它不是负数，就打印一条提示。

```c
#include <stdio.h>
int main(void)
{
    int a;
    scanf("%d", &a);
    if (a >= 0)
    {
        printf("非负数");    // 屏幕上出现
    }
    return 0;
}
```

运行结果

输入 5 → 输出 非负数

输入 0 → 输出 非负数

输入 -3 → 什么都不输出

三步走

第一步：scanf 把数读进 a。

第二步：if 判断 a 是否大于等于 0。

第三步：成立才执行 printf。

02

深入理解：结构与控制

本部分讲什么 · 花括号管到哪、嵌套怎么写、阶梯按什么顺序走

哈爾濱工程大学

花括号与缩进

关键 if 后面不带花括号时，它只管得住紧跟的那一条语句。

```
// 错例：缩进骗了眼睛
if (a > 0)
    printf("正数");
    printf("总会执行");   // 不受 if 管
// 输入 -1 时，第二句照样打印
```

```
// 正解：加花括号打包
if (a > 0)
{
    printf("正数");
    printf("也受 if 管");
}
// 输入 -1 时，两句都不打印
```

记住：不带花括号只管到第一个分号；要管多条语句，必须加花括号。

嵌套 if ：选择里再选择

```
// 先问及格没，及格了再问高不高
int score;
scanf("%d", &score);
if (score >= 60)
{
    if (score >= 90)
        printf("优秀");     // 90 分及以上
    else
        printf("及格");     // 60 到 89 分
}
else
    printf("不及格");       // 60 分以下
```

选择里再选择

在一个 if 的内部再写一个完整的 if ，形成两层判断。

外层成立才进内层

外层不成立时，内层连看都不看，直接跳到外层 else 。

缩进体现层级

每往里一层，向右缩进 4 个空格，人一眼看出谁包着谁。

else if 阶梯：多选一

重点

```
//  五档成绩：条件从上往下排成一排
if (score >= 90)
    printf("优秀");
else if (score >= 80)
    printf("良好");
else if (score >= 70)
    printf("中等");
else if (score >= 60)
    printf("及格");
else
    printf("不及格");
```

整条链只出一个结果

五个分支里最多执行一个，绝不会出现两个结果。

else if 是合写

else 后面紧跟一个 if ，中间不能再插别的语句。

最后的 else 兜底

前面都不成立时，剩下的情况全部交给它。

阶梯的执行顺序

```
// 假设 score 输入的是 85
if (score >= 90)
    printf("优秀");
else if (score >= 80)
    printf("良好");
else if (score >= 70)
    printf("中等");
else if (score >= 60)
    printf("及格");
else
    printf("不及格");
```

`score >= 90`	85 不满足，往下看
`score >= 80`	满足，输出良好
`score >= 70`	不再判断
`score >= 60`	不再判断
`else`	不执行

从上往下逐条判断，第一个成立就执行并收工，后面的一律不看。

嵌套与阶梯怎么选

哈尔滨工程大学

```
// 写法一：嵌套，一层层往里钻
if (x > 0)
{
    if (x > 100) printf("大正数");
    else printf("小正数");
}
else printf("非正数");
// x 取 50 时，输出 小正数
```

```
// 写法二：阶梯，一排条件往下排
if (x > 100) printf("大正数");
else if (x > 0) printf("小正数");
else printf("非正数");
// x 取 50 时，输出 小正数
// 同样的结果，少了两个花括号
// 层级更浅，出错的机会也更少
```

用阶梯

条件是同一把尺子的不同档位，一排写完，从上往下读最清楚

用嵌套

先分大类，再在大类里细分，比如先看及格，再看优秀

别套三层

能排成一排就别往里套，套得越深，else 越难配对

悬空 else 问题

坑 else 的缩进是写给人看的，编译器根本不看缩进。

```
// 看着配外层，其实配的是内层
if (a > 0)
    if (b > 0)
        printf("都大于 0");
else
    printf("a 不大于 0");  // 想错了

// 实际上：a 大于 0 且 b 小于等于 0
// 时，才会打印 a 不大于 0
```

```
// 加花括号，归属立刻清楚
if (a > 0)
{
    if (b > 0)
        printf("都大于 0");
}
else
    printf("a 不大于 0");  // 配外层
```

配对规则：else 只认离它最近、还没配对的 if ；拿不准就加花括号。

03

综合运用：六个经典判断

本部分讲什么 · 正负零、比大小、奇偶、等级、闰年、分段函数

哈爾濱工程大學

判断正负零

场景 1 读入一个整数，判断它是正数、负数还是零。

```
//  三种情况，正好一条三级的阶梯
int x;
scanf("%d", &x);
if (x > 0)
    printf(" 正数 ");
else if (x < 0)
    printf(" 负数 ");
else
    printf(" 零 ");
```

运行结果

输入 7 → 输出 正数

输入 -2 → 输出 负数

输入 0 → 输出 零

要点

三种情况互斥，只会命中一个。

前两个条件都不成立， x 只能是 0 。

所以最后的 else 不用再写条件。

两个数比大小

场景 2 读入两个整数，打印其中较大的那个。

```
// 只比一次，就能分出高下
int a, b;
scanf("%d%d", &a, &b);
if (a > b)
    printf("max = %d", a);   // a 更大
else
    printf("max = %d", b);   // b 更大或相等
```

运行结果

输入 3 9 → max = 9

输入 8 2 → max = 8

输入 5 5 → max = 5

要点

两个数比较，一次判断够用。

相等时 a 大于 b 不成立，走 else 。

相等时输出谁都一样，不用再补条件。

奇偶判断

场景 3 读入一个整数，判断它是奇数还是偶数。

```
// 除以 2 看余数：余数是 0 就是偶数
int n;
scanf("%d", &n);
if (n % 2 == 0)
    printf("偶数");
else
    printf("奇数");

// 百分号是取余，取余优先于判等
```

运行结果

输入 8 → 输出 偶数

输入 7 → 输出 奇数

输入 0 → 输出 偶数

要点

取余算出余数，余数为 0 即整除。

判等一定用两个等号，别写成赋值。

0 除以 2 余 0，所以 0 也是偶数。

成绩等级：阶梯的经典

```
int score;
scanf("%d", &score);
if (score >= 90)
    printf("A");
else if (score >= 80)
    printf("B");
else if (score >= 70)
    printf("C");
else if (score >= 60)
    printf("D");
else
    printf("E");
```

score >= 90	等级 A
80 ~ 89 分	等级 B
70 ~ 79 分	等级 C
60 ~ 69 分	等级 D
60 分以下	等级 E

边界别漏

90、80、70、60 这些分数要带等号

闰年判断：两层 if

不用合并条件

哈尔滨工程大学

```
int year;
scanf("%d", &year);
if (year % 100 == 0)
{
    if (year % 400 == 0) printf(" 闰年 ");
    else printf(" 平年 ");
}
else
{
    if (year % 4 == 0) printf(" 闰年 ");
    else printf(" 平年 ");
}
```

整百年

能被 400 整除 → 闰年

整百年

不能被 400 整除 → 平年

非整百年

能被 4 整除 → 闰年

非整百年

不能被 4 整除 → 平年

分段函数求值

```
// 按 x 所在区间挑公式
int x, y;
scanf("%d", &x);
if (x < 1)
    y = x;
else if (x < 10)
    y = 2 * x - 1;
else
    y = 3 * x - 11;
printf("y = %d", y);
```

第一段

x 小于 1 ：y 等于 x

第二段

x 小于 10 ：y 等于 2 乘 x 减 1

第三段

其余：y 等于 3 乘 x 减 11

阶梯自带下界：能走到第二个条件，说明 x 已经不小于 1 ，不用再写一遍。

例题精讲（一）

哈爾濱工程大學

例 1 输入一个整数，判断它是奇数还是偶数。

```
#include <stdio.h>
int main(void)
{
    int n;
    scanf("%d", &n);
    if (n % 2 == 0)
        printf("偶数");
    else
        printf("奇数");
    return 0;
```

运行结果

输入 8 → 偶数

输入 7 → 奇数

输入 0 → 偶数

解题思路

用取余算出除以 2 的余数。

余数为 0 走 if，否则走 else。

0 除以 2 余 0，也是偶数。

例题精讲（二）

例 2 输入三个整数，输出最大值。要求只用嵌套 if ，不许把条件合并。

```
int a, b, c, max;
scanf("%d%d%d", &a, &b, &c);
if (a > b)
{
    if (a > c) max = a;
    else max = c;
}
else
{
    if (b > c) max = b;
    else max = c;
}
printf("max = %d", max);
```

运行结果

输入 3 9 5 → max = 9

输入 7 2 8 → max = 8

输入 4 4 4 → max = 4

解题思路

第一层：先让 a 和 b 比出胜者。

第二层：胜者再和 c 比一次。

两条支路都写全，覆盖所有情况。

例题精讲（三）

例 3 输入百分制成绩，90 以上 A，80 以上 B，70 以上 C，60 以上 D，其余 E。

```
// 省去 #include 与主函数骨架，其余同例 1
int score;
scanf("%d", &score);
if (score >= 90) printf("A");
else if (score >= 80) printf("B");
else if (score >= 70) printf("C");
else if (score >= 60) printf("D");
else printf("E");
```

运行结果

输入 95 → A

输入 85 → B

输入 60 → D

解题思路

条件从高到低排，别倒着写。

每个边界都要带等号。

最后用 else 兜住不及格。

例题精讲（四）

哈爾濱工程大學

例 4 输入年份，判断它是闰年还是平年（用两层 if 写）。

```
int year;
scanf("%d", &year);
if (year % 100 == 0)
{
    if (year % 400 == 0) printf(" 闰年 ");
    else printf(" 平年 ");
}
else
{
    if (year % 4 == 0) printf(" 闰年 ");
    else printf(" 平年 ");
}
```

运行结果

输入 2000 → 闰年

输入 1900 → 平年

输入 2024 → 闰年

解题思路

第一层：按是否整百年分两类。

第二层：整百年看 400，其余看 4。

两条路都写 else，不会有遗漏。

例题精讲（五）

哈爾濱工程大學

例 5 出租车：3 公里以内 10 元，超出部分每公里 2 元，按整数公里算。

```
// 省去 #include 与主函数骨架，其余同例 1
int km, money;
scanf("%d", &km);
if (km <= 3)
    money = 10;
else
    money = 10 + (km - 3) * 2;
printf("%d 元", money);
```

运行结果

输入 2 → 10 元

输入 3 → 10 元

输入 8 → 20 元

解题思路

先找分界点：3 公里。

不超过就只收起步价 10 元。

超出则起步价加超出部分的钱。

常见错误盘点

哈尔滨工程大学

- ✗ 把判等 `==` 写成赋值 `=`，条件永远成立，编译器还不报错
- ✗ 忘写花括号，以为两条语句都受控，其实只管第一条
- ✗ else 后面又加条件，或者顺手多写一个分号
- ✗ 照搬数学写法连用关系符：`0 <= x <= 10` 在 C 里不是区间判断
- ✗ 嵌套不缩进，程序能跑，但几天后自己也看不懂谁包着谁
- ✗ 整数比较用 `==` 没问题，实数带误差，别直接判相等

小结

if 让程序在半路分岔：条件成立走一支，不成立走另一支；多个条件排成 else if 阶梯，需要细分就写嵌套 if 。

下节预告

下节课的 && 和 || 两个运算符，能把层层嵌套压成一条判断。

哈爾濱工程大學